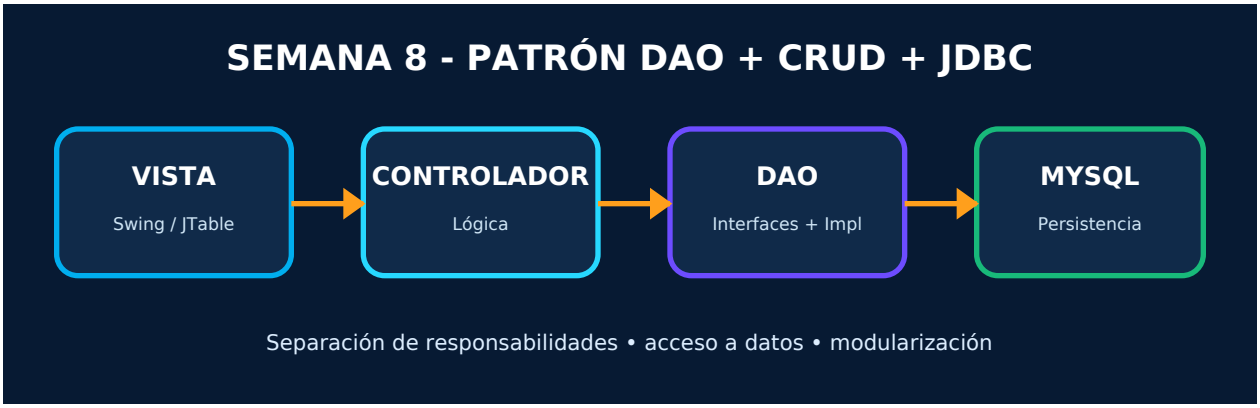


GUÍA DE EJERCICIOS RESUELTOS

Desarrollo Orientado a Objetos II (PRY2203)

Semana 8: Gestionando datos mediante operaciones CRUD

Prof. Miguel Puebla
Santiago de Chile
3 de octubre de 2026

Base académica: Semana 8 de la guía de aprendizaje de la Experiencia 3: patrón DAO, interfaces DAO, DAOImpl, PreparedStatement, ResultSet, validación, excepciones y modularización.

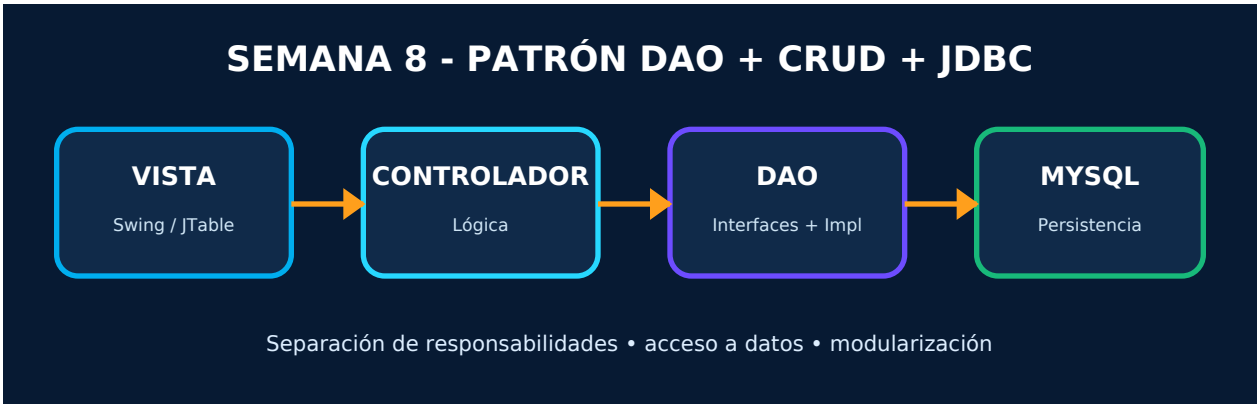
Fuentes oficiales de apoyo: documentación oficial de MySQL Connector/J y API JDBC de Java/Oracle.

Referencia sugerida:
Puebla, M. (2026). Guía de ejercicios resueltos: Semana 8 - Gestionando datos mediante operaciones CRUD. Desarrollo Orientado a Objetos II, Duoc UC. Santiago de Chile.

Índice

1. Síntesis de contenidos de la Semana 8	3
2. ProductoDAO: contrato CRUD	4
3. ProductoDAOImpl: insertar	5
4. Buscar producto con ResultSet	6
5. Actualizar producto	7
6. Eliminar producto	8
7. UsuarioDAO y autenticación	9
8. CategoriaDAO y listado	10
9. VentaDAO y registro de venta	11
10. Validación desde la interfaz	12
11. Flujo completo Vista-Controlador-DAO	13
12. Referencias y forma de citación	14

1. Síntesis de contenidos de la Semana 8



La Semana 8 aplica el patrón DAO para separar el acceso a datos de la interfaz gráfica y los controladores. Cada entidad puede disponer de una interfaz DAO que define operaciones y una implementación DAOImpl que ejecuta JDBC contra MySQL.

- Interfaces DAO para Producto, Usuario, Categoría, Venta y DetalleVenta.
- Implementaciones DAOImpl con JDBC.
- Operaciones CRUD.
- PreparedStatement y ResultSet.
- Manejo de SQLException.
- Validación previa desde la interfaz.
- Modularización y reutilización.



2. ProductoDAO: contrato CRUD



Enunciado

Definir el contrato de persistencia para Producto.

Solución propuesta

```
public interface ProductoDAO {
  void insertar(Producto producto);
  void actualizar(Producto producto);
  void eliminar(int idProducto);
  Producto buscarPorId(int idProducto);
  List<Producto> listarTodos();
  List<Producto> buscarPorNombre(String nombre);
}
```

Explicación

La interfaz declara operaciones sin incluir SQL. ProductoDAOImpl implementará este contrato.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

3. ProductoDAOImpl: insertar



Enunciado

Registrar un producto mediante PreparedStatement.

Solución propuesta

```
public void insertar(Producto p) throws SQLException {
  String sql =
      "INSERT INTO productos(nombre,id_categoria,stock,valor)"
    + " VALUES(?,?,?,?)";

  try (PreparedStatement ps = conexion.prepareStatement(sql)) {
    ps.setString(1, p.getNombre());
    ps.setInt(2, p.getIdCategoria());
    ps.setInt(3, p.getStock());
    ps.setInt(4, p.getValor());
    ps.executeUpdate();
  }
}
```

Explicación

El INSERT queda encapsulado en DAOImpl y los valores se asignan como parámetros.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

4. Buscar producto con ResultSet



Enunciado

Buscar por ID y transformar la fila en un objeto.

Solución propuesta

```
public Producto buscarPorId(int id) throws SQLException {
  String sql = "SELECT * FROM productos WHERE id_producto=?";
  try (PreparedStatement ps = conexion.prepareStatement(sql)) {
    ps.setInt(1, id);
    try (ResultSet rs = ps.executeQuery()) {
      if (rs.next()) {
        return new Producto(
          rs.getInt("id_producto"),
          rs.getString("nombre"),
          rs.getInt("id_categoria"),
          rs.getInt("stock"),
          rs.getInt("valor"));
      }
    }
  }
  return null;
}
```

Explicación

ResultSet entrega las columnas recuperadas y permite construir el objeto Producto.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

5. Actualizar producto



Enunciado

Modificar un registro existente.

Solución propuesta

```
public void actualizar(Producto p) throws SQLException {
  String sql =
      "UPDATE productos SET nombre=?, id_categoria=?,"
    + " stock=?, valor=? WHERE id_producto=?";
  try (PreparedStatement ps = conexion.prepareStatement(sql)) {
    ps.setString(1, p.getNombre());
    ps.setInt(2, p.getIdCategoria());
    ps.setInt(3, p.getStock());
    ps.setInt(4, p.getValor());
    ps.setInt(5, p.getIdProducto());
    ps.executeUpdate();
  }
}
```

Explicación

El ID identifica el registro; el resto de los parámetros contienen los nuevos valores.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

6. Eliminar producto



Enunciado

Eliminar por identificador.

Solución propuesta

```
public void eliminar(int idProducto) throws SQLException {
  String sql = "DELETE FROM productos WHERE id_producto=?";
  try (PreparedStatement ps = conexion.prepareStatement(sql)) {
    ps.setInt(1, idProducto);
    ps.executeUpdate();
  }
}
```

Explicación

La vista puede confirmar la acción y luego entregar el ID al DAO.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

7. UsuarioDAO y autenticación



Enunciado

Delegar la autenticación a la capa DAO.

Solución propuesta

```
public Usuario autenticar(String correo, String clave)
    throws SQLException {
  String sql =
      "SELECT * FROM usuarios "
    + "WHERE correo=? AND contraseña=? AND activo=true";
  try (PreparedStatement ps = conexion.prepareStatement(sql)) {
    ps.setString(1, correo);
    ps.setString(2, clave);
    try (ResultSet rs = ps.executeQuery()) {
      if (rs.next()) return mapearUsuario(rs);
    }
  }
  return null;
}
```

Explicación

El formulario entrega credenciales y UsuarioDAO consulta la base; la vista no contiene SQL.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

8. CategoriaDAO y listado



Enunciado

Listar categorías para un JComboBox.

Solución propuesta

```
public List<Categoria> listarTodas() throws SQLException {
  List<Categoria> lista = new ArrayList<>();
  String sql =
      "SELECT id_categoria,nombre_categoria FROM categorias";
  try (PreparedStatement ps = conexion.prepareStatement(sql);
       ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
      lista.add(new Categoria(
        rs.getInt("id_categoria"),
        rs.getString("nombre_categoria")));
    }
  }
  return lista;
}
```

Explicación

El DAO devuelve objetos Categoria y la vista usa la colección para cargar el componente.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

9. VentaDAO y registro de venta



Enunciado

Registrar una venta y recuperar su ID generado.

Solución propuesta

```
public int registrarVenta(Venta venta) throws SQLException {
  String sql = "INSERT INTO ventas(total) VALUES(?)";
  try (PreparedStatement ps = conexion.prepareStatement(
      sql, Statement.RETURN_GENERATED_KEYS)) {
    ps.setDouble(1, venta.getTotal());
    ps.executeUpdate();
    try (ResultSet rs = ps.getGeneratedKeys()) {
      if (rs.next()) return rs.getInt(1);
    }
  }
  return -1;
}
```

Explicación

El identificador generado puede utilizarse después para relacionar los detalles de la venta.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

10. Validación desde la interfaz



Enunciado

Validar antes de enviar datos al DAO.

Solución propuesta

```
private void guardarProducto() {
  String nombre = txtNombre.getText().trim();
  if (nombre.isEmpty()) {
    mostrarError("Ingrese el nombre del producto");
    return;
  }
  int stock = (int) spStock.getValue();
  if (stock < 0) {
    mostrarError("El stock no puede ser negativo");
    return;
  }
  controlador.guardarProducto(leerFormulario());
}
```

Explicación

La interfaz evita enviar información inválida a la capa de persistencia.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

11. Flujo completo Vista-Controlador-DAO



Enunciado

Integrar las capas para guardar y recargar productos.

Solución propuesta

```
// Vista
btnGuardar.addActionListener(e -> controlador.guardar());

// Controlador
public void guardar() {
  Producto p = vista.leerProducto();
  try {
    productoDAO.insertar(p);
    vista.cargarProductos(productoDAO.listarTodos());
  } catch (SQLException ex) {
    vista.mostrarError("Error al acceder a la base de datos");
  }
}
```

Explicación

La vista captura el evento; el controlador coordina; DAO persiste y la vista se actualiza.

Comprobación: verifica que la interfaz no contenga SQL y que el acceso a datos quede delegado al DAO.

12. Referencias y forma de citación

Fuente académica principal

Duoc UC. (2026). Guía de aprendizaje: Conectando interfaz gráfica y base de datos. Experiencia 3. Desarrollo Orientado a Objetos II. Semana 8: Gestionando datos mediante operaciones CRUD.

Fuentes técnicas oficiales de apoyo

Oracle / Java. API JDBC (java.sql): Connection, PreparedStatement, ResultSet y SQLException.

Oracle MySQL. MySQL Connector/J Developer Guide.

Referencia sugerida de esta guía

Puebla, M. (2026). Guía de ejercicios resueltos: Semana 8 - Gestionando datos mediante operaciones CRUD. Desarrollo Orientado a Objetos II, Duoc UC. Santiago de Chile.

Cita parentética: (Puebla, 2026)
Cita narrativa: Puebla (2026)

